

Hot Air

Fleet assignment is a hard and critical problem in airline management. Hot air is a simple case of this class of problem.

Problem 1

- Maximize the profit, i.e. the ticket sales subtracted by the flying cost

Sets

- Cities: $c \in Cities = \{ "Brownsville", "Dallas", "Austin", "ElPaso" \}$
- Time periods: $t \in Time = \{1, 2, 3\}$

Parameters

- Passenger demand: $D_{t,c1,c2}$: The number of passengers flying from city $c1$ to $c2$ in timeperiod t
- Ticket price: $P_{c1,c2}$. Paid for each passenger flying from city $c1$ to city $c2$
- Take off cost: $TOC_{c1,c2}$. Cost of flying from city $c1$ to city $c2$ independent of the number of passengers.
- Number of of hangar spots available: H_c . I.e. number of of planes which can stay over-night in city c
- Plane capacity: $PlaneCapacity = 120$

Decision variables

- N.o. planes flying from city $c1$ to city $c2$ in time period t : $y_{t,c1,c2} \in \{0, 1, 2, 3, 4\}$.
- N.o. passengers flying from city $c1$ to city $c2$ in time period t in total: $x_{t,c1,c2} \geq 0$

Note: The variables $y_{t,c,c}$ will be used for keeping track of planes that stay in a city, c , without flying in time period t . For this to work it is a requirement that $TOC_{c,c} = 0 \forall c$ and $D_{t,c,c} = 0 \forall c, t$ since these should not impact the objective value.

Model

Objective:

- Maximize profit (income from ticket sales minus expense of take-off):

$$\sum_{t,c1,c2} P_{c1,c2} \cdot x_{t,c1,c2} - \sum_{t,c1,c2} TOC_{c1,c2} \cdot y_{t,c1,c2}$$

Constraints:

- Ensure that not more passengers are transported than the demand:

$$x_{t,c1,c2} \leq D_{t,c1,c2} \forall c1, c2, t$$

- Ensure that there are seats to all passengers transported:

$$x_{t,c1,c2} \leq PlaneCapacity \cdot y_{t,c1,c2} \forall c1, c2, t$$

- Ensures that the correct number of planes stay overnight in the various cities, i.e. take off from the city in time period 1:

$$\sum_{c2} y_{1,c1,c2} = H_{c1} \forall c1$$

- Balance constraint: The number of planes which arrive to a city in the previous time period **has** to take off in the next time period:

$$\sum_{c2} y_{t,c2,c1} = \sum_{c2} (y_{t+1,c1,c2} | (t < T) + y_{1,c1,c2} | (t = T)) \forall c1, t$$

Recall, that for a plane to **stay** in a city, it simply takes-off and lands in the same city, not carrying passengers and costing nothing. This is some-what of a trick, which makes the model elegant.

The full model in Julia/JuMP, available with the name

`HotAir1.jl`

from the book web-site, is given below:

```

*****
# HotAir Assignment, Question 1, "Mathematical Programming Modelling" (42112)
using JuMP
using HiGHS
*****

# PARAMETERS
cities = ["Brownsville", "Dallas", "Austin", "El Paso"]
C=length(cities)
time = [1,2,3]
T=length(time)

# ticket-prices ticket_price[cf,ct]
P=[
    0  99  89 139;
    109  0  99 169;
    109 104  0 129;
    159 149 119  0]

# takeoff-cost takeoff_cost[cf,ct]
TOC=[
    0  5100  4400  8000;
    5100  0 11200  6900;
    4400 11200  0  5700;
    8000  6900  5700  0]

# passenger demand passengers[t,cf,ct]
D=zeros{Int16,T,C,C}
D[1,:,:]=
[ # timeperiod 1
    0  50  53  14 ;
    84  0  80  21 ;
    17  58  0  40 ;
    31  79  34  0
]
D[2,:,:]=
[ # timeperiod 2
    0  15  53  52 ;
    17  0 134  29 ;
    24 128  0  99 ;
    23  15  30  0
]
D[3,:,:]=
[ # timeperiod 3
    0  3  16  9 ;

```

```

48    0 104  48 ;
62   92   0  68 ;
13   15   21   0
]
# plane starting places
H=[2 1 1 0]
PlaneCapacity=120
*****

# Model
hotair = Model(HiGHS.Optimizer)

# passengers transported in time-period t from city cf to city ct
@variable(hotair, x[t=1:T,cf=1:C,ct=1:C] >= 0)

# planes flying in timeperiod t from city cf to city ct
@variable(hotair, 0 <= y[1:T,cf=1:C,ct=1:C] <= 4, Int)

# Maximize profit
@objective(hotair, Max,
            sum( P[cf,ct]*x[t,cf,ct] for t=1:T, cf=1:C, ct=1:C if cf != ct) -
            sum( TOC[cf,ct]*y[t,cf,ct] for t=1:T, cf=1:C, ct=1:C if cf != ct) )

# demand limit
@constraint(hotair, [t=1:T,cf=1:C,ct=1:C],
            x[t,cf,ct] <= D[t,cf,ct] )

# plain capacity limit
@constraint(hotair, [t=1:T,cf=1:C,ct=1:C],
            x[t,cf,ct] <= PlaneCapacity*y[t,cf,ct] )

# where the planes over-night
@constraint(hotair, [cf=1:C],
            sum( y[1,cf,ct] for ct=1:C) == H[cf] )

# plane balance, what goes in must come out
@constraint(hotair, [t=1:T,c1=1:C],
            sum( y[t,c2,c1] for c2=1:C) == sum( (t<T ? y[t+1,c1,c2] : y[1,c1,c2]) for c2=1:C) )
*****

# solve
optimize!(hotair)
println("Termination status: $(termination_status(hotair))")

```

```

#####

#####

# Report results
println("-----");
if termination_status(hotair) == MOI.OPTIMAL
    println("RESULTS:")
    println("objective = $(objective_value(hotair))")
    yres=round.(Int64,value.(y))
    xres=round.(Int64,value.(x))
    for t=1:T
        println("Time step: $(t)")
        for cf=1:C # from
            for ct=1:C # to
                if yres[t,cf,ct]>0
                    println(" Plane: $(yres[t,cf,ct])      "*
                        "Passengers: $(xres[t,cf,ct])  $(cities[cf])  - > $(cities[ct])")
                end
            end
        end
        println("")
    end
else
    println(" No solution")
end
println("-----");
#####

```

Comments to the Julia/JuMP program:

- Notice: Again we use the conditional if-statement in a constraint to differentiate between time being less than T and being equal to T

Hot Air considers to build hangars in other cities. Which cities should they choose and how much can they earn more? Note that this is a relaxation of the requirement that the number of planes staying overnight should match the current hangar capacities.

Problem 2

- Maximize the profit, when allowing the 4 planes to stay overnight in the most profitable cities

New Decision variables

- Number of hangar spots in city c : $v_c \in \{0, 1, 2, 3, 4\}$

New and updated constraints:

- Ensures that the correct number of planes stay overnight, i.e. take off from a city in time period 1:

$$\sum_{c2} y_{1,c1,c2} = v_{c1} \quad \forall c1$$

- Ensure that exactly 4 hangars are built:

$$\sum_c v_c = 4$$

Notice, this is actually a relaxation and we can achieve a higher profit, but we of course do not consider the costs of establishing the hangars.

The full model in Julia/JuMP, available with the name

HotAir2.jl

from the book web-site, is given below:

```
*****
# HotAir Assignment, Question 2, "Mathematical Programming Modelling" (42112)
using JuMP
using HiGHS
*****

# PARAMETERS
cities = ["Brownsville", "Dallas", "Austin", "El Paso"]
C=length(cities)
time = [1,2,3]
T=length(time)

# ticket-prices P[cf,ct]
P=[
    0  99  89 139;
  109  0  99 169;
  109 104  0 129;
  159 149 119  0]
```

```

# takeoff cost TOC[cf,ct]
TOC=[
    0  5100  4400  8000;
5100    0 11200  6900;
4400 11200    0  5700;
8000  6900  5700    0]

# passenger demand D[t,cf,ct]
D=zeros(Int16,T,C,C)
D[1,:,:]=
[ # timeperiod 1
  0  50  53  14 ;
84   0  80  21 ;
17  58   0  40 ;
31  79  34   0
]
D[2,:,:]=
[ # timeperiod 2
  0  15  53  52 ;
17   0 134  29 ;
24 128   0  99 ;
23  15  30   0
]
D[3,:,:]=
[ # timeperiod 3
  0   3  16   9 ;
48   0 104  48 ;
62  92   0  68 ;
13  15  21   0
]

# plane starting places
H=[2 1 1 0]
PlaneCapacity=120
#####

#####
# Model
hotair = Model(HiGHS.Optimizer)

# passengers transported in time-period t from city cf to city ct
@variable(hotair, x[t=1:T,cf=1:C,ct=1:C] >= 0)

# planes flying in timeperiod t from city cf to city ct
@variable(hotair, 0 <= y[t=1:T,cf=1:C,ct=1:C] <= 4, Int)

# planes overnighing in city c

```

```

@variable(hotair, 0 <= v[c=1:C] <= 4, Int)

# Maximize profit
@objective(hotair, Max,
    sum( P[cf,ct]*x[t,cf,ct] for
        t=1:T, cf=1:C, ct=1:C if cf != ct) -
    sum( TOC[cf,ct]*y[t,cf,ct] for
        t=1:T, cf=1:C, ct=1:C if cf != ct) )

# demand limit
@constraint(hotair, [t=1:T,cf=1:C,ct=1:C],
    x[t,cf,ct] <= D[t,cf,ct] )

# plain capacity limit
@constraint(hotair, [t=1:T,cf=1:C,ct=1:C],
    x[t,cf,ct] <= PlaneCapacity*y[t,cf,ct] )

# where the planes over-night
@constraint(hotair, [cf=1:C],
    sum( y[1,cf,ct] for ct=1:C) == v[cf] )

# limit the number of overnights
@constraint(hotair, sum( v[c] for c=1:C) == 4 )

# plane balance, what goes in must come out
@constraint(hotair, plane_balance[t=1:T,c1=1:C],
    sum( y[t,c2,c1] for c2=1:C) ==
    sum( (t<T ? y[t+1,c1,c2] : y[1,c1,c2]) for c2=1:C) )
#####

#####
# solve
optimize!(hotair)
println("Termination status: $(termination_status(hotair))")
#####

#####
# Report results
println("-----");
if termination_status(hotair) == MOI.OPTIMAL
    println("RESULTS:")
    println("objective = $(objective_value(hotair))")
    yres=round.(Int64,value.(y))
    xres=round.(Int64,value.(x))
    for t=1:T

```



```

println("Time step: $(t)")
for cf=1:C # from
    for ct=1:C # to
        if yres[t,cf,ct]>0
            println(" Plane: $(yres[t,cf,ct])      "*
                "Passengers: $(xres[t,cf,ct])  $(cities[cf])  - > $(cities[ct])")
        end
    end
end
println("")
end
else
    println(" No solution")
end
println("-----");
#####

```

Management wants a print-out of the profit on each route. This is actually **not** a question of changing the model or obtaining a new solution, but just extracting more information from the solution you can already get, thus the only changes are in the post-processing and printing of results (after `optimize!()` is called)

Problem 3

The full model in Julia/JuMP, available with the name

`HotAir3.jl`

from the book web-site, is given below:

```

#####
# HotAir Assignment, Question 3, "Mathematical Programming Modelling" (42112)
using JuMP
using HiGHS
#####

#####
# PARAMETERS
cities = ["Brownsville", "Dallas", "Austin", "El Paso"]
C=length(cities)
time = [1,2,3]
T=length(time)

# ticket-prices P[cf,ct]

```

```

P=[
    0  99  89 139;
    109 0  99 169;
    109 104 0 129;
    159 149 119 0]

# takeoff cost TOC[cf,ct]
TOC=[
    0  5100  4400  8000;
    5100  0 11200  6900;
    4400 11200  0  5700;
    8000  6900  5700  0]

# passenger demand D[t,cf,ct]
D=zeros(Int16,T,C,C)
D[1,:,:]=
[ # timeperiod 1
    0  50  53  14 ;
    84  0  80  21 ;
    17  58  0  40 ;
    31  79  34  0
]
D[2,:,:]=
[ # timeperiod 2
    0  15  53  52 ;
    17  0 134  29 ;
    24 128  0  99 ;
    23  15  30  0
]
D[3,:,:]=
[ # timeperiod 3
    0  3  16  9 ;
    48  0 104  48 ;
    62  92  0  68 ;
    13  15  21  0
]

# plane starting places
H=[2 1 1 0]
PlaneCapacity=120
*****

*****

# Model
hotair = Model(HiGHS.Optimizer)

# passangers transported in time-period t from city cf to city ct

```

```

@variable(hotair, x[t=1:T,cf=1:C,ct=1:C] >= 0)

# planes flying in timeperiod t from city cf to city ct
@variable(hotair, 0 <= y[1:T,1:C,1:C] <= 4, Int)

# planes overnighing in city c
@variable(hotair, 0 <= v[1:C] <= 4, Int)

# Maximize profit
@objective(hotair, Max,
    sum( P[cf,ct]*x[t,cf,ct] for
        t=1:T, cf=1:C, ct=1:C if cf != ct) -
    sum( TOC[cf,ct]*y[t,cf,ct] for
        t=1:T, cf=1:C, ct=1:C if cf != ct) )

# demand limit
@constraint(hotair, [t=1:T,cf=1:C,ct=1:C],
    x[t,cf,ct] <= D[t,cf,ct] )

# plain capacity limit
@constraint(hotair, [t=1:T,cf=1:C,ct=1:C],
    x[t,cf,ct] <= PlaneCapacity*y[t,cf,ct] )

# where the planes over-night
@constraint(hotair, [cf=1:C],
    sum( y[1,cf,ct] for ct=1:C) == v[cf] )

# limit the number of overnights
@constraint(hotair, sum( v[c] for c=1:C) == 4 )

# plane balance, what goes in must come out
@constraint(hotair, [t=1:T,c1=1:C],
    sum( y[t,c2,c1] for c2=1:C) ==
    sum( (t<T ? y[t+1,c1,c2] : y[1,c1,c2]) for c2=1:C) )
#####

#####

# solve
optimize!(hotair)
println("Termination status: $(termination_status(hotair))")
#####

#####

# Report results
println("-----");

```

```

if termination_status(hotair) == MOI.OPTIMAL
    println("RESULTS:")
    println("objective = $(objective_value(hotair))")
    yres=round.(Int64,value.(y))
    xres=round.(Int64,value.(x))
    for t=1:T
        println("Time step: $(t)")
        for i=1:C # from
            for j=1:C # to
                if yres[t,i,j]>0

print( " Plane: $(yres[t,i,j])\t Passengers: $(xres[t,i,j])\t")
print( "$(cities[i]) - > $(cities[j])")
print( "\t Profit: $((xres[t,i,j]*P[i,j])-TOC[i,j])\t")
print( "Take off Cost: $(TOC[i,j])")
println( "\t Total Ticket sale: $(xres[t,i,j]*P[i,j])")
                end
            end
        end
    end
else
    println(" No solution")
end
println("-----");
#####

```

The management now forces you to ensure that only profitable flights are flown. This ignores the network effect, i.e. that we may fly a plane at a loss from one city to another to make a larger profit from that other city in the next time period. The only thing necessary is to add an extra constraint for each city pair and time period.

Problem 4

New constraints:

- Calculate the profit for **each**, flight and force profitability of that flight:

$$P_{i,j} \cdot x_{t,i,j} - TOC_{i,j} \cdot y_{t,i,j} \geq 0 \quad \forall i, j, t$$

The full model in Julia/JuMP, available with the name

HotAir4.jl

from the book web-site, is given below:

```

*****
# HotAir Assignment, Question 4, "Mathematical Programming Modelling" (42112)
using JuMP
using HiGHS
*****

# PARAMETERS
cities = ["Brownsville", "Dallas", "Austin", "El Paso"]
C=length(cities)
time = [1,2,3]
T=length(time)

# ticket-prices P[cf,ct]
P=[
    0  99  89 139;
    109  0  99 169;
    109 104  0 129;
    159 149 119  0]

# takeoff cost TOC[cf,ct]
TOC=[
    0  5100  4400  8000;
    5100  0 11200  6900;
    4400 11200  0  5700;
    8000  6900  5700  0]

# passenger demand D[t,cf,ct]
D=zeros(Int16,T,C,C)
D[1,:,:]=
[ # timeperiod 1
    0  50  53  14 ;
    84  0  80  21 ;
    17  58  0  40 ;
    31  79  34  0
]
D[2,:,:]=
[ # timeperiod 2
    0  15  53  52 ;
    17  0 134  29 ;
    24 128  0  99 ;
    23  15  30  0
]
D[3,:,:]=
[ # timeperiod 3
    0  3  16  9 ;

```

```

48    0 104  48 ;
62   92   0  68 ;
13   15   21   0
]
# plane starting places
H=[2 1 1 0]
PlaneCapacity=120
*****

# Model
hotair = Model(HiGHS.Optimizer)

# passengers transported in time-period t from city cf to city ct
@variable(hotair, x[t=1:T,cf=1:C,ct=1:C] >= 0)

# planes flying in timeperiod t from city cf to city ct
@variable(hotair, 0 <= y[t=1:T,cf=1:C,ct=1:C] <= 4, Int)

# planes overnighiting in city c
@variable(hotair, 0 <= v[1:C] <= 4, Int)

# Maximize profit
@objective(hotair, Max,
    sum( P[cf,ct]*x[t,cf,ct] for
        t=1:T, cf=1:C, ct=1:C if cf != ct) -
    sum( TOC[cf,ct]*y[t,cf,ct] for
        t=1:T, cf=1:C, ct=1:C if cf != ct) )

# demand limit
@constraint(hotair, [t=1:T,cf=1:C,ct=1:C],
    x[t,cf,ct] <= D[t,cf,ct] )

# plain capacity limit
@constraint(hotair, [t=1:T,cf=1:C,ct=1:C],
    x[t,cf,ct] <= PlaneCapacity*y[t,cf,ct] )

# where the planes over-night
@constraint(hotair, [cf=1:C],
    sum( y[1,cf,ct] for ct=1:C) == v[cf] )

# limit the number of overnights
@constraint(hotair, sum( v[c] for c=1:C) == 4 )

# plane balance, what goes in must come out

```

```

@constraint(hotair, [t=1:T,c1=1:C],
sum( y[t,c2,c1] for c2=1:C) ==
sum( (t<T ? y[t+1,c1,c2] : y[1,c1,c2]) for c2=1:C) )

# only fly profitable tours
@constraint(hotair, [t=1:T,i=1:C,j=1:C],
P[i,j]*x[t,i,j] -
TOC[i,j]*y[t,i,j] >= 0)
#####

#####
# solve
optimize!(hotair)
println("Termination status: $(termination_status(hotair))")
#####

#####
# Report results
println("-----");
if termination_status(hotair) == MOI.OPTIMAL
    println("RESULTS:")
    println("objective = $(objective_value(hotair))")
    yres=round.(Int64,value.(y))
    xres=round.(Int64,value.(x))
    for t=1:T
        println("Time step: $(t)")
        for i=1:C # from
            for j=1:C # to
                if yres[t,i,j]>0

print( " Plane: $(yres[t,i,j])\t Passengers: $(xres[t,i,j])\t")
print( "$ (cities[i]) - > $(cities[j])")
print( "\t Profit: $((xres[t,i,j]*P[i,j])-TOC[i,j])\t")
print( "Take off Cost: $(TOC[i,j])")
println( "\t Total Ticket sale: $(xres[t,i,j]*P[i,j])")
                end
            end
        end
    end
else
    println(" No solution")
end
println("-----");
#####

```

Introducing discount tickets is now studied. This is an advanced problem which

is quite hard to solve. A new variable is necessary

Problem 5

This final part of the assignment is rather tricky and not that important for the course, but included to pose a challenge for the ambitious student.

- Maximize the profit including both the income from the normal tickets and the discount tickets.

New Decision variables

- Number of discount passengers transported from city $c1$ over city cm to city $c2$: $0 \leq u_{t,c1,cm,c2} \leq 480 \in \mathbb{Z}^+$

Model

Objective:

- Maximize profit:

$$\begin{aligned} & \sum_{t,c1,c2} P_{c1,c2} \cdot x_{t,c1,c2} + \\ & \sum_{t,c1,cm,c2} 0.7 \cdot P_{c1,c2} \cdot u_{t,c1,cm,c2} - \\ & \sum_{t,c1,c2} TOC_{c1,c2} \cdot y_{t,c1,c2} \end{aligned}$$

Constraints:

- Ensure that there are seats to all passengers transported:

$$\begin{aligned} & x_{t,c1,c2} + \\ & \sum_c u_{t,c1,c2,c} + \\ & \sum_c u_{t-1,c,c1,c2} | (t > 1) \\ & \leq PlaneCapacity \cdot y_{t,c1,c2} \quad \forall c1, c2, t \end{aligned}$$

- Ensure that not more discount passengers are transported than the demand:

$$\sum_c u_{t,c1,c,c2} \leq 0.5 \cdot D_{t,c1,c2} \quad \forall c1, c2, t$$

Notice the way the discount passengers are ensured seats on the planes.

The full model in Julia/JuMP, available with the name

HotAir5.jl

from the book web-site, is given below:

```
#####
# HotAir Assignment, Question 5, "Mathematical Programming Modelling" (42112)
using JuMP
using HiGHS
#####

#####
# PARAMETERS
cities = ["Brownsville", "Dallas", "Austin", "El Paso"]
C=length(cities)
time = [1,2,3]
T=length(time)

# ticket-prices P[cf,ct]
P=[
    0  99  89 139;
    109  0  99 169;
    109 104  0 129;
    159 149 119  0]

# takeoff cost TOC[cf,ct]
TOC=[
    0  5100  4400  8000;
    5100  0 11200  6900;
    4400 11200  0  5700;
    8000  6900  5700  0]

# passenger demand D[t,cf,ct]
D=zeros{Int16,T,C,C}
D[1,:,:]=
[ # timeperiod 1
    0  50  53  14 ;
    84  0  80  21 ;
```

```

    17  58   0  40 ;
    31  79  34   0
    ]
D[2, :, :] =
[ # timeperiod 2
  0  15  53  52 ;
  17   0 134  29 ;
  24 128   0  99 ;
  23  15  30   0
]
D[3, :, :] =
[ # timeperiod 3
  0   3  16   9 ;
  48   0 104  48 ;
  62  92   0  68 ;
  13  15  21   0
]
# plane starting places
H = [2 1 1 0]
PlaneCapacity = 120
*****

#*****

# Model
hotair = Model(HiGHS.Optimizer)

# passengers transported in time-period t from city i to city j
@variable(hotair, x[t=1:T, cf=1:C, ct=1:C] >= 0)
for t=1:T
    for c=1:C
        set_upper_bound(x[t, c, c], 0)
    end
end

# planes flying in timeperiod t from city i to city j
@variable(hotair, 0 <= y[t=1:T, cf=1:C, ct=1:C] <= 4, Int)

# planes overnighing in city c
@variable(hotair, 0 <= v[c=1:C] <= 4, Int)

# passengers transported in time-period t from city cf to city cm
# (city midt) and in timepeiod t+1 from city cm to city ct
@variable(hotair, 0 <= u[t=1:T, cf=1:C, cm=1:C, ct=1:C] <= 480, Int)
for t=1:T
    for c1=1:C
        for c2=1:C

```

```

        set_upper_bound(u[t,c1,c1,c2],0)
        set_upper_bound(u[t,c1,c2,c2],0)
    end
end
end
for cf=1:C
    for cm=1:C
        for ct=1:C
            set_upper_bound(u[3,cf,cm,ct],0)
        end
    end
end
end

# Maximize profit
@objective(hotair, Max,
    sum( P[cf,ct]*x[t,cf,ct] for t=1:T, cf=1:C, ct=1:C) +
    sum( 0.7*P[cf,ct]*u[t,cf,cm,ct]
        for t=1:T, cf=1:C, cm=1:C, ct=1:C) -
    sum( TOC[cf,ct]*y[t,cf,ct] for t=1:T, cf=1:C, ct=1:C)
)

# demand limit
@constraint(hotair, [t=1:T,cf=1:C,ct=1:C],
    x[t,cf,ct] <= D[t,cf,ct] )

# discount demand limit
@constraint(hotair, [t=1:T,cf=1:C,ct=1:C],
    sum(u[t,cf,cm,ct] for cm=1:C) <= 0.5*D[t,cf,ct] )

# plain capacity limit
@constraint(hotair, [t=1:T,cf=1:C,ct=1:C],
    x[t,cf,ct] + sum( u[t,cf,ct,c] for c=1:C) +
    (t>1 ? (sum( u[t-1,c,cf,ct] for c=1:C)) : 0) <=
    PlaneCapacity*y[t,cf,ct] )

# where the planes over-night, i.e. starting from in the morning
@constraint(hotair, [cf=1:C],
    sum( y[1,cf,c] for c=1:C) == v[cf] )

# limit the number of overnights
@constraint(hotair, sum( v[c] for c=1:C) == 4 )

# plane balance, what goes in must come out
@constraint(hotair, [t=1:T,c1=1:C],
    sum( y[t,c2,c1] for c2=1:C) ==
    sum( (t<T ? y[t+1,c1,c2] : y[1,c1,c2]) for c2=1:C) )

```

```

#####

#####
# solve
optimize!(hotair)
println("Termination status: $(termination_status(hotair))")
#####

#####
# Report results
println("-----");
if termination_status(hotair) == MOI.OPTIMAL
    println("RESULTS:")
    println("objective = $(objective_value(hotair))")
    yres=round.(Int64,value.(y))
    xres=round.(Int64,value.(x))
    ures=round.(Int64,value.(u))
    for t=1:T
        println("Time step: $(t)")
        for cf=1:C # from
            for ct=1:C # to
                if yres[t,cf,ct]>0
                    first=sum( ures[t,cf,ct,k] for k=1:C)
                    second=sum( ures[t,k,cf,ct] for k=1:C)

                    print( " Plane: $(yres[t,cf,ct]) \t")
                    print( " Passengers: $(xres[t,cf,ct]) first: $(first)  " *
                        "second: $(second) \t")
                    print( " $(cities[cf]) - > $(cities[ct]) \t ")
                    print( " Profit: $((xres[t,cf,ct]*P[cf,ct]) - TOC[cf,ct]) \t " *
                        "Take off Cost: $(TOC[cf,ct]) \t")
                    println( " Total Ticket sale: $((xres[t,cf,ct]*P[cf,ct]) )" )
                end
            end
        end
    end
else
    println(" No solution")
end
println("-----");
#####

#####
println("Successfull end of $(PROGRAM_FILE)")
#####

```

Comments: For performance we have set all the $x_{t,c,c}$ variables (corresponding to passengers transported on planes that do not fly) to 0. This is not necessary as they will have to be 0 in any feasible solution due to the demand constraint (all $D_{t,c,c} = 0$), but can help the solver. Similarly it is not necessary to set and upper bound on the u -variables of 480, but since at most 4 planes with a capacity of 120 passengers can fly between cities at the same time, then they can never exceed this value, so again we help the solver by providing information to narrow down the search for a feasible solution.